Gabby Football Quiz Web Application - Technical Specification Document (Draft v4)

Project: Gabby Quiz App | **Target Stack:** Node.js 24 LTS + Express 5 + Replit PostgreSQL | **Status:** READY TO BUILD (0% GUESSWORK)

1. Executive Summary & System Overview

The Gabby Football Quiz App is an interactive educational web platform engineered for football enthusiasts and high school students. Built using a robust monolithic Node.js/Express architecture with a relational PostgreSQL backend, the application enables registered players to test their football knowledge across five specialized categories. The platform incorporates single-question card navigation, instant correct/incorrect visual feedback, auto-submitting countdown timers, historical attempt tracking, and real-time student leaderboards. Teachers and administrators have elevated permissions to manage questions directly via a Web GUI editor and execute bulk CSV imports through a row-validated preview interface.

2. Quiz Content & Football Topic Domains

The platform features exactly five active quizzes with at least ten questions each. Every question provides exactly four answer choices (A, B, C, D) and strictly one correct answer:

1. **NFL History & Origins:** League founding, AFL-NFL merger history, iconic team origins, and early rule evolution.
2. **Super Bowl Highlights:** Championship games, MVP performances, scoring records, and dynasty lore.
3. **Rules, Positions & Scoring:** Penalties, field geometry, offensive/defensive player roles, and scoring mechanics (touchdowns, safeties, field goals).
4. **All-Time Greats:** Hall of Fame quarterbacks, defensive icons, rushing leaders, and career statistical milestones.
5. **College Football & Trophies:** Heisman trophy legends, bowl game traditions, rivalry history, and championship lore.

3. Custom Functional Features

Custom Feature 1: Instant Feedback & Explanations

Enhances learning by providing real-time visual results and context for every question selected during an active quiz attempt:

- **Visual Feedback:** Selected answer option immediately turns green (if correct) or red (if incorrect). The correct answer button always highlights green.
- **Selection Lock:** All four choice buttons immediately lock upon selection to prevent answer alteration. Navigating back with the "Previous" button preserves the locked state and visual feedback.
- **Contextual Explanation:** Displays detailed background text clarifying why the correct answer is right below the options.

Custom Feature 2: Timed Quiz Mode (Auto-Submit)

Introduces optional competitive pressure with strict timing enforcement per quiz session:

- **Countdown Clock:** Sticky 5-minute (300 seconds) countdown timer displayed prominently at the top right of the quiz card view.
- **Auto-Submit Logic:** When time hits 0:00, JavaScript locks selection, aggregates answered items, submits the attempt to PostgreSQL, and redirects to the results screen.
- **Timestamp Integrity:** Server verifies start and submit timestamps to prevent client manipulation.

4. User Experience, Navigation & Workflows

Quiz Navigation & Completion Rules

- **Single-Question Card Navigation:** Questions render one at a time inside a centered card container with a "Question X of 10" indicator and progress bar.
- **Controls:** Includes a "Previous" button (disabled on item 1), a "Next" button (unlocked upon answer selection), and a "Submit Quiz" button on item 10.
- **Unfinished Attempts:** Abandoned or unfinished attempts (e.g., navigating away before final submit) are discarded and never recorded in the database.
- **Results Screen:** Upon submission, the user sees their raw score (e.g., 8 / 10), percentage correct (e.g., 80%), and a prominent "Retry Quiz" button directly below the score card to restart the session.

Authentication & Accounts

- **User Management:** Registration, Login, and Logout forms.
- **Logout Action:** A "Log Out" button in the header bar triggers a server request to

terminate the active Express session, clear session cookies, and redirect to the Login page.

- **Password Security:** Passwords must be hashed using bcrypt prior to database storage and are never saved as plain text.

Leaderboard Specifications

Leaderboard Type	Primary Ranking Metric	Displayed Fields	Tie-Breaking Logic
Overall Leaderboard	Total completed quiz attempts	Username, Total Completed Attempts	Earliest timestamp of the most recent completed attempt
Per-Quiz Leaderboard	Highest score on specific quiz	Username, High Score (out of 10), Total Quiz Attempts	Fewest attempts to achieve high score, then earliest timestamp

5. Admin Question Editor Specification

- **Access Control:** The "Question Editor" navigation link renders exclusively for users logged in with role = 'admin'. Server middleware enforces session verification, returning 403 Forbidden for unauthorized users.
- **Question Bank List View:**
  - Displays section header with a **"+ Add New Question"** button.
  - Dropdown filter to view questions by specific quiz title or category.
  - Data table displaying: Question ID, Assigned Quiz Name, Question Text preview, Correct Option, and Action buttons ("Edit" and "Delete").
- **Add / Edit Question Form:**
  - **Target Quiz:** Dropdown selector populated from the quizzes table (quiz_id).
  - **Question Text:** Required multi-line text input (question_text).
  - **Answer Choices:** Four separate required text inputs labeled Choice A (option_a), Choice B (option_b), Choice C (option_c), and Choice D (option_d).
  - **Correct Option:** Radio button group or selector restricted strictly to 'A', 'B', 'C', or 'D' (correct_option).
  - **Explanation:** Required multi-line text input for answer explanation (explanation).

- **Form Actions & Workflows:**
  - **Saving:** "Save Question" executes client validation, sending POST (new) or PUT (edit) requests to Express. Success returns to list view with confirmation.
  - **Deleting:** "Delete" displays a modal confirmation ("Are you sure you want to delete this question?") before issuing a DELETE API request.

6. Admin CSV Import Specification & Validation Pipeline

Teachers can upload or paste CSV files to bulk seed or update question banks. The parser enforces a strict **all-or-nothing** validation pipeline prior to database committing:

```
quiz_id, question_text, option_a, option_b, option_c, option_d, correct_option, explanation
1, "Who won Super Bowl I?", "Packers", "Chiefs", "Cowboys", "Patriots", "A", "Green Bay won 35-10 in 1967."
```

- **Header Check:** Validates exact column header match on line 1. Missing or misspelled headers reject the entire file.
- **Row Integrity:** Identifies empty text fields or invalid quiz_id foreign keys.
- **Option Constraint:** Flags any row where correct_option is not strictly 'A', 'B', 'C', or 'D'.
- **Preview & Import Policy:** Previews valid rows in green and invalid rows in red with exact line numbers and error descriptions. If *any* row is invalid, the "Commit Import" button remains disabled until all errors are fixed in the source file.

7. Technical Architecture & Relational Schema

Layer	Technology	Implementation Rule
Frontend	Plain HTML5, CSS3, ES6+ JavaScript	Vanilla fetch API calls; no client frameworks (React/Vue).
Server	Node.js 24 LTS & Express 5	Monolithic server serving static assets and dynamic REST JSON endpoints.

Layer	Technology	Implementation Rule
Database	Replit PostgreSQL (pg client)	Parameterized SQL queries exclusively to eliminate SQL injection.
Auth / Security	bcrypt password hashing	Passwords hashed prior to persistence; session cookie auth.

Relational Schema Design

Table Name	Columns & Data Types	Constraints & Keys
users	id (SERIAL), username (VARCHAR), email (VARCHAR), password_hash (TEXT), role (VARCHAR), created_at (TIMESTAMP)	PRIMARY KEY (id), UNIQUE (username, email), role IN ('player', 'admin')
quizzes	id (SERIAL), title (VARCHAR), category (VARCHAR), description (TEXT), created_at (TIMESTAMP)	PRIMARY KEY (id), UNIQUE (title)
questions	id (SERIAL), quiz_id (INT), question_text (TEXT), option_a (TEXT), option_b (TEXT), option_c (TEXT), option_d (TEXT), correct_option (CHAR(1)), explanation (TEXT), created_at (TIMESTAMP)	PRIMARY KEY (id), FOREIGN KEY (quiz_id) REFERENCES quizzes(id) ON DELETE CASCADE

Table Name	Columns & Data Types	Constraints & Keys
attempts	id (SERIAL), user_id (INT), quiz_id (INT), score (INT), total_questions (INT), completed_at (TIMESTAMP)	PRIMARY KEY (id), FOREIGN KEY (user_id) REFERENCES users(id), FOREIGN KEY (quiz_id) REFERENCES quizzes(id)

8. Five-Phase Structured Development Roadmap

1. **Phase 1: Foundation & DB Setup:** Initialize Express server, establish Replit PostgreSQL connection pool with pg, execute database table migration scripts, and configure static asset hosting.
2. **Phase 2: Authentication & RBAC:** Build register/login/logout API routes with bcrypt hashing, Express session management, and role-based client views (Player vs. Admin UI rendering).
3. **Phase 3: Core Quiz Engine & Instant Feedback:** Implement single-question card navigation UI, instant feedback visual styling (green/red + explanation text), 5-minute countdown timer with auto-submit, and seed 5 football quizzes with 10+ questions each.
4. **Phase 4: History & Leaderboards:** Construct attempt persistence API, "My History" user view displaying percentages, Overall Leaderboard (total attempts), and Per-Quiz Leaderboards (high score + attempt counts) with tie-breaker logic.
5. **Phase 5: Admin Tools & Deployment:** Develop GUI Question Editor, CSV parser with all-or-nothing row validation & preview UI, perform full QA testing in Replit Workspace, and deploy to Replit Production.